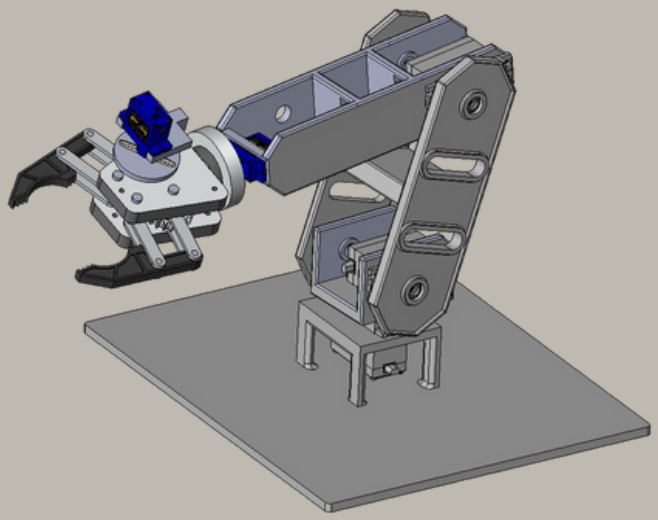
Semi - Automated
Robotic Arm
GE107 Tinkering Lab (G16_WED)

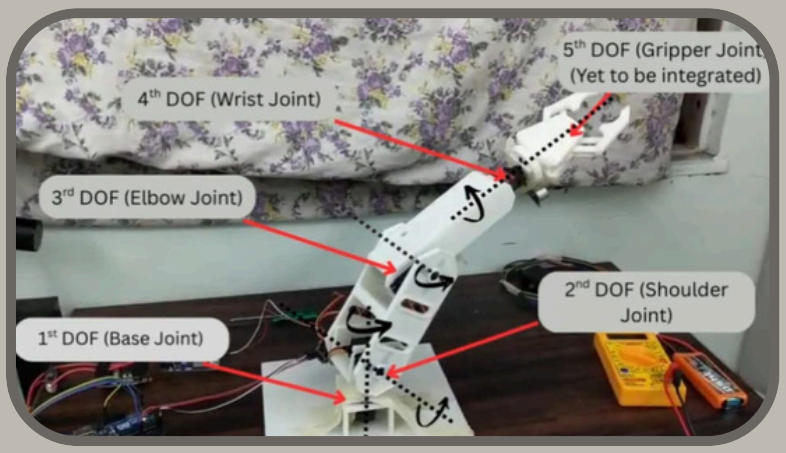
The objective of this project was to design and develop a semi-automated robotic arm capable of performing basic pick-and-place operations using sensor feedback and programmable control logic. This project has taken inspiration from the robotic arm present in automobile factories

List Of Components Used

- Arduino UNO
- 4 Servo Motors
- 12V DC Source
- Buck Converter (To step down 12V to 5V)
- HC-05 Bluetooth Module
- PCA9685
- Push Button Switches
- PLA (to print the frame)



CAD model of the Final design



The arm consists of a base joint, shoulder joint, elbow joint, wrist joint, and a gripper joint (not implemented yet). Each joint provides certain movements that capture the flexibility of an arm, such that there can be accurate control and a vast amount of motion.

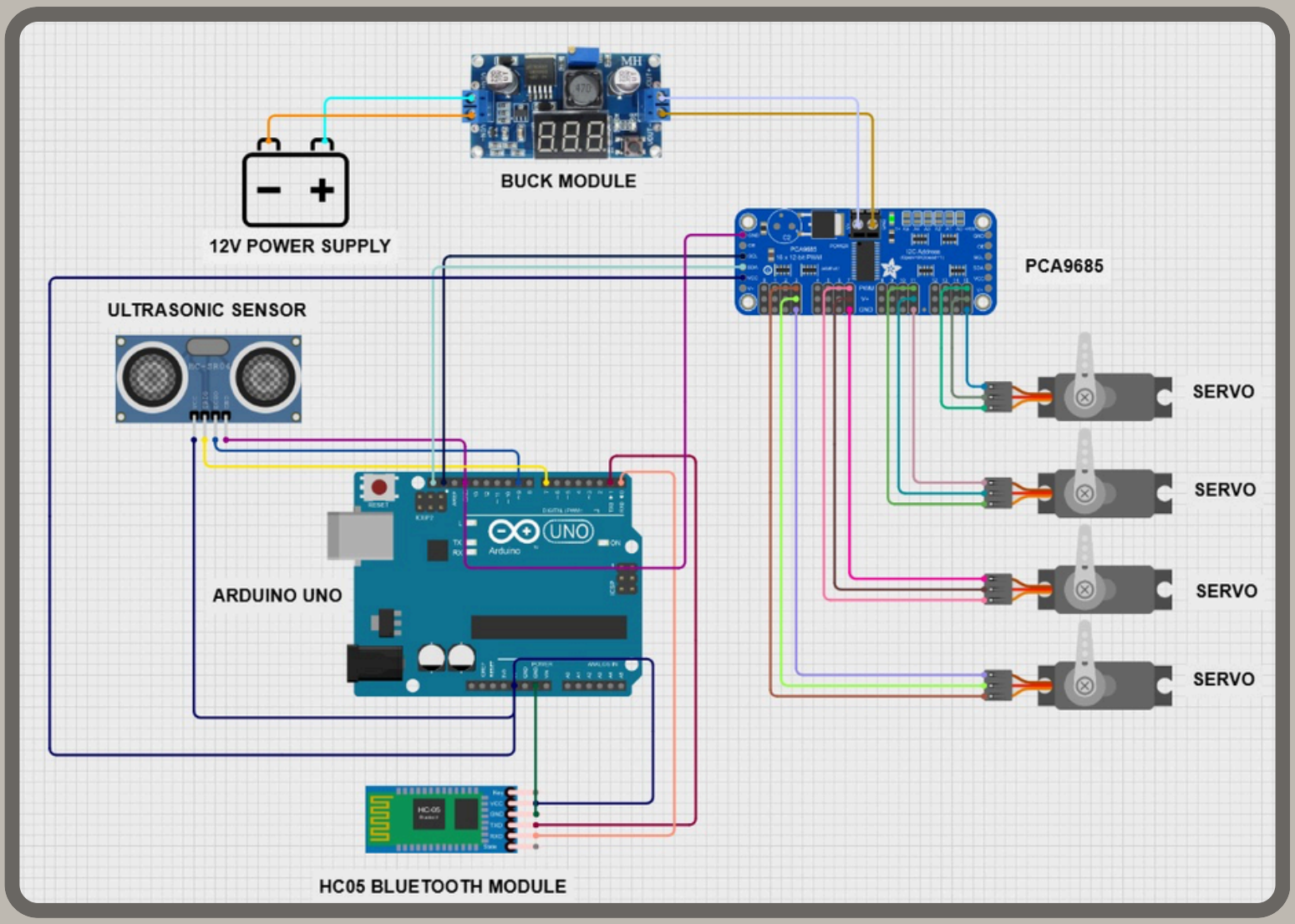
The robotic arm's forward kinematics calculates the position of the end-effector based on given joint angles, allowing us to predict where the arm will reach in space. In contrast, inverse kinematics determines the joint angles required to move the end-effector to a specific target, enabling precise and automated motion control. To enhance environmental awareness, we integrated an ultrasonic sensor that measures distances to nearby objects. This allows the arm to detect obstacles, maintain safe distances, and effectively approach objects for tasks such as picking, placing, or sorting.

We developed predefined kinematic paths for tasks like pick-and-place by mapping precise joint angle sequences. In the absence of real-time feedback, motion was executed through calibrated, time-based actuation.

A notable feature of the system is its Bluetooth integration with the Blynk app, which serves both as a feedback interface—indicating when the arm comes into contact with an object—and as a remote control for executing predefined actions like picking or dropping items.

Importantly, the robotic arm is fully self-designed and 3D printed by our team. The accuracy and precision in its assembly reflect the careful planning and detailed modeling we carried out during the design phase.

CIRCUIT DIAGRAM



Labelled Circuit Digagram Of all the electronic used in the project

- **Arduino Uno acts as the central controller, managing input signals and controlling servo motors through the PCA9685 module.**
- **PCA9685 is a 16-channel PWM driver that powers and controls the position of the connected servo motors precisely.**
- **4 Servo Motors are connected to PCA9685, each controlling a specific joint in the semi-automated robotic arm.**
- **A 12V DC source provides the main power supply, stepped down for safe use by other components using a buck converter.**
- **Buck Converter reduces 12V to 5V, powering the PCA9685 board and servo motors with sufficient and safe voltage.**
- **SCL and SDA lines connect PCA9685 to Arduino, enabling I2C communication for controlling servo positions programmatically.**
- **Ultrasonic Sensor is connected to Arduino for distance measurement, potentially enabling obstacle detection or auto-stop features.**
- **HC-05 Bluetooth Module connects to Arduino's TX and RX pins, allowing wireless control of the robotic arm via a custom developed mobile application.**
- **Common Ground Connections are established between all modules to ensure stable and reliable voltage references for all components.**
- **The setup enables semi-automated operation, combining programmed motion with remote manual control for versatile robotic arm movement.**

Challenges Faced

1- Servo Angle Calibration Error-

During our project, we faced a servo calibration issue where setting servos to 0 degrees didn't match the expected position. The variation was due to mechanical offsets and PWM differences. We solved it by manually adjusting servo horns and applying code-level offsets, ensuring accurate and repeatable movements.

2- Ultrasonic Sensor Placement Error-

We initially placed the ultrasonic sensor in a position where wrist movements affected object detection. To fix this, we repositioned the sensor below the claw, ensuring consistent and accurate distance measurement regardless of the arm's movement or orientation.

We also experienced fluctuations in distance readings from the ultrasonic sensor due to environmental noise and surface reflectivity. To address this, we applied signal averaging, introduced a noise-tolerant threshold, and carefully positioned the sensor to reduce cross-interference.

3- Fast movement of servo-

Initially, we faced issues with the servo motors moving too quickly to their target positions, resulting in jerky and imprecise arm motion. Sudden servo jumps not only made the system unstable but also caused lot of trouble in calibration.

To address this, we implemented smooth servo motion in our code by gradually updating the servo angles one degree at a time toward their target using a controlled delay. This was achieved through a moveSmoothDegrees() function that incrementally adjusts each servo's position with a consistent delay (stepDelay) between steps. This allowed for smooth and synchronized arm movements, improving both accuracy and calibration in initial phase.

4- Lack of Real time feedback

Without encoders, it was hard to track the actual position of joints, leading to a discrepancy between commanded and actual motion. We mitigated this by estimating positions through calibrated time-based motion and adding limit switches for end-point calibration.

5- Predefining the angles for intended motion

We faced challenges in determining the exact sequence of angles required to move the robotic arm to a desired position or follow a specific path. Since the arm lacked inverse kinematics capabilities at initial stages, we couldn't compute joint angles automatically for a given end-effector position. As a result, we had to manually define intermediate joint angles to achieve smooth and precise movements. This required breaking down the overall path into smaller steps and fine-tuning each joint's movement based on trial-and-error and visual feedback.

Future Improvements

1. Automatic search of the object in the proximity using ultrasonic sensors.

2. Using App as complete remote control for the servo angles and adding a toggle switch to select between manual and position control.

3. Setting predefined motion path using the app itself, gives a lot of flexibility.

4. A complete gripping mechanism which can hold drop various object precisely.